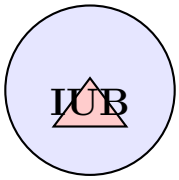

INDEPENDENT UNIVERSITY

An Undergraduate Internship Project on

Orient Computer E-commerce Website

By

Sultana Tasnim Riza

Student ID: 2111329

Summer 2026

Supervisor:

Bijoy Rahman Arif

Senior Lecturer

Department of Computer Science & Engineering

Independent University, Bangladesh

12 September, 2026

Dissertation submitted in partial fulfillment for the degree of Bachelor of Science in Computer Science

Department of Computer Science & Engineering

Independent University, Bangladesh

Attestation

I, **Sultana Tasnim Riza**, ID: **2111329**, hereby declare that the report titled **"Orient Computer E-commerce Website"** submitted in partial fulfillment of the requirement for the Degree of Bachelor of Science in Computer Science is the result of my own work. I also confirm that the work was carried out as a student of Independent University, Bangladesh.

I would like to acknowledge the guidance of my supervisor, Bijoy Rahman Arif, who supported me throughout this project. I also declare that this work represents my personal understanding and the outcomes I achieved during my internship period.

The report also properly cites and credits all sources used for further inquiry about this report, contacting my supervisor in the internship setting, Md Sabbir Hasan Khan at Orient Computers through official communication channels.

Signature	Date

Sultana Tasnim Riza
Name

i

Acknowledgement

I am grateful for the opportunity to complete this internship and produce the project described in this report. My thanks go first to the Almighty and to my family, who were patient and encouraging throughout the long hours of software development, debugging, and report writing.

My academic supervisor, **Bijoy Rahman Arif**, provided clear direction and asked the right questions at each review milestone. Without those check-ins the project would have drifted in several unhelpful directions.

I also thank the team at **Orient Computers**, particularly **Md Sabbir Hasan Khan**, for explaining the business requirements honestly, including the complex domain workflows: how orders are fulfilled, how district-wise delivery charges operate in Bangladesh, and how manual bKash and Nagad payment verifications are handled. His mentorship motivated me to surpass expectations.

I express my sincere gratitude to the faculty members of the Department of Computer Science & Engineering at Independent University, Bangladesh for establishing strong foundational concepts in web application design, data structures, and database management systems.

ii

Letter of Transmittal

12 September, 2026

Bijoy Rahman Arif
Senior Lecturer
Department of Computer Science & Engineering, School of Engineering and Computer Science
Independent University, Bangladesh

Subject: Submission of an internship report for graduation completion

Dear Sir,

I am submitting my Internship Report as part of the Bachelor Program in Computer Science. I am grateful for your supervision during my internship at Orient Computers, where I worked for three months under the guidance of Md Sabbir Hasan Khan.

The project involved designing and building a full-stack e-commerce web application for Orient Computer, a computer hardware and electronics retailer based in Dhaka, Bangladesh. The application lets customers browse products, place orders, and pay using Cash on Delivery, bKash, or Nagad. An administrative dashboard allows staff to manage the full order lifecycle including payment verification, product cataloging, stock control, and localized shipping fees.

I have documented the architectural decisions made, technical challenges encountered, and solutions engineered. If any section requires further clarification, I am pleased to discuss it.

Thank you for your valuable guidance and consideration.

Yours sincerely,

Sultana Tasnim Riza
ID: 2111329
Department of Computer Science & Engineering,
Independent University, Bangladesh

iii

Evaluation Committee

Supervision Panel

.....	.....
Academic Supervisor	Industry Supervisor

Panel Members

.....	.....
Panel Member 1	Panel Member 2
.....	.....
Panel Member 3	Panel Member 4

Office Use

.....	.....
Internship Coordinator	Head of the Department

iv

Abstract

This report documents the design and implementation of a full-stack e-commerce web platform for Orient Computer, a prominent retailer in Dhaka, Bangladesh, specializing in PC components, UPS/IPS power backup systems, industrial batteries, solar equipment, and office technology. Prior to this project, the enterprise operated solely via brick-and-mortar showrooms and phone inquiries, lacking an integrated digital sales and order management channel.

The system is constructed upon a three-tier architecture utilizing the MERN stack (MongoDB Atlas, Express.js, React 18 with Vite, and Node.js) styled with Tailwind CSS. The customer-facing frontend facilitates dynamic catalog browsing, multi-criteria filtering, parametric search, persistent cart migration, and a multi-step checkout workflow. Tailored to local business context, the application natively supports Cash on Delivery (COD) as well as manual Mobile Financial Services (MFS) via bKash and Nagad with transaction proof submission.

A dedicated administration portal empowers staff to manage catalog hierarchies, stock levels, district-wise delivery charges, promotional coupons, customer reviews, and order fulfillment states. Security measures include bcrypt password hashing, stateless JSON Web Token (JWT) authorization, role-based access control, input sanitization, and strict server-side price/stock validation to prevent tampering. Comprehensive unit, integration, and security testing confirm the robustness, responsiveness, and operational viability of the software solution.

Keywords: MERN Stack, E-commerce, Bangladesh, bKash, Nagad, React, Node.js, Express.js, MongoDB Atlas, JWT Authentication, Role-Based Access Control.

v

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Overview and Background

Practical internship experience plays an essential role in bridging the gap between theoretical classroom learning and real-world engineering problem solving. As part of the graduation requirements for the Bachelor of Science in Computer Science at Independent University, Bangladesh, this three-month internship was completed at Orient Computers.

Orient Computers is an established technology and electronics provider based in Dhaka, Bangladesh. The company's official website lists product categories including PC components, renewable energy products, UPS, IPS, batteries, telecom equipment, audio-visual products, and office equipment [2]. Historically, business operations relied on physical retail interactions and manual phone consultations.

Transitioning retail operations into the digital domain within the Bangladeshi market presents unique engineering and infrastructural considerations. International card payment gateways exhibit limited penetration, necessitating direct support for Mobile Financial Services (MFS) like bKash and Nagad alongside Cash on Delivery (COD). Furthermore, shipping logistics vary considerably between inside Dhaka metropolitan zones and outer districts. This project aims to engineer an end-to-end e-commerce solution addressing these specific operational needs.

1.2 Objectives

The primary objectives of the internship project were to:

1. **Design and Develop a Responsive Customer Storefront:** Create an intuitive interface allowing customers to search, filter, inspect technical specifications, and manage shopping carts across desktop and mobile viewports.

2. **Implement Localized Payment & Checkout Processing:** Build a secure checkout pipeline calculating dynamic delivery fees by district and handling COD, bKash, and Nagad payments.

3. **Create an Administrative Operations Dashboard:** Provide store administrators with tools for inventory tracking, category hierarchy management, customer oversight, coupon management, and order status updates.

4. **Engineer a Payment Verification Workflow:** Implement administrative review functionality for manual MFS transactions with screenshot validation and state tracking.

5. **Enforce Enterprise-Grade Security and Integrity:** Employ bcrypt password hashing, stateless JWT authentication, role-based route guards, and server-side price freezing to prevent client-side tampering.

1.3 Scope

The scope of the project encompasses:

- User authentication, profile management, and persistent shipping addresses.
- Product catalog with full-text indexing, multi-level categorization, brand filtering, and real-time stock indicators.
- Guest cart management in browser storage with automated server-side synchronization upon login.
- Order creation with atomic stock decrement and server-side price verification.
- Anonymous order tracking by order tracking number and phone number.
- Customer product reviews and wishlist management.
- Full-featured admin panel with analytics, low-stock warnings, and transaction moderation.

Out-of-scope items include third-party automated merchant aggregator APIs (e.g., SSLCommerz), automated SMS gateways, and interactive PC builder compatibility tools, which are reserved for subsequent release phases.

Chapter 2

Literature Review

2.1 Relationship with Undergraduate Studies

The Orient Computer e-commerce platform is not only an industrial software artifact; it is also a practical application of several undergraduate computing concepts. The project required requirements analysis, interface design, database modeling, API construction, authentication, and security controls. These areas connect directly with the academic preparation received throughout the Bachelor of Science in Computer Science program.

- **CSE309 (Web Application Development):** The course concepts of client-server communication, asynchronous JavaScript, RESTful API design, and component-based frontend construction directly influenced the React and Express implementation. The use of React for reusable interface components and Express for route-based server logic follows the design guidance commonly documented for these technologies [9, 10].

- **CSE307 (System Analysis and Design):** Requirements gathering, use-case modeling, rich pictures, and software requirements specification (SRS) practices were applied before implementation. This was important because Orient Computer's workflow involved both customer-facing shopping behavior and internal administrative operations.

- **CSE303 (Database Management):** Although the project uses MongoDB rather than a relational database, database design knowledge remained essential. Entity relationships, normalization principles, indexing, and query design helped determine when information should be embedded in a document and when it should be referenced through separate collections. Mongoose was used to enforce schema rules and validation at the application layer [11].

- **CSE211 (Object-Oriented Programming):** Modular programming concepts shaped the separation of backend models, controllers, middleware, and utility functions. This separation reduced duplication and made the codebase easier to test, maintain, and extend.

- **CSE403 (Information Security):** Security coursework informed the use of password hashing, JWT-based authentication, role-based access control, server-side validation, and rate limiting. These decisions align with OWASP guidance on common web application risks such as broken access control, injection, and authentication weaknesses [8].

2.2 E-Commerce Context in Bangladesh

E-commerce adoption in Bangladesh has grown alongside wider internet access, smartphone usage, and digital payment awareness. Government digital transformation initiatives have encouraged online service delivery and digital transactions, while private retailers have responded by building web-based sales channels [1]. For a computer and electronics retailer such as Orient Computer, an online platform must support more than a simple product list. Customers expect searchable catalogues, accurate technical specifications, visible stock information, delivery options, and payment methods that reflect local habits.

Payment behavior is a particularly important design factor in the Bangladeshi market. Cash on Delivery remains familiar to many customers, while Mobile Financial Services (MFS) such as bKash and Nagad are widely used for person-to-person and merchant payments. Bangladesh Bank recognizes MFS as part of the country's payment infrastructure, and local retailers commonly combine mobile payments with branch, delivery, or order-verification processes [3]. This context influenced the decision to include COD, bKash, and Nagad rather than relying only on international card-based checkout flows.

2.3 Related Works and Technical Landscape

Existing Bangladeshi technology retailers provide useful reference points for this project. Star Tech presents itself as a major computer, laptop, and gadget e-commerce and retail shop in Bangladesh, with online shopping support for desktops, laptops, office equipment, components, accessories, and related products [4]. Ryans Computers similarly operates an e-commerce platform and publishes payment method information that includes cash, bKash, Nagad, Rocket, and card options [5]. These competitors show that successful technology retail platforms in Bangladesh generally need deep category navigation, specification-heavy product pages, search and filtering, and locally appropriate payment support.

The Orient Computer platform adapts these market-validated patterns to the company's own product mix. Unlike general consumer marketplaces, the system must represent UPS, IPS, batteries, solar products, computer hardware, networking devices, and accessories. Such products often require technical attributes, model numbers, warranty details, and stock status. Therefore, the catalogue design emphasizes structured product specifications, category and brand filters, full-text search, and administrative inventory control.

2.4 Existing Website Analysis

An analysis of the official Orient Computers & Engineering website helped identify strengths that should be preserved and gaps that the internship project could address [2]. The existing website already presents Orient as a wholesale and retail technology store in Bangladesh and shows a wide product range across PC components, AVR, renewable energy, UPS, IPS, batteries, telecom products, audio-visual products, and office equipment. Its category structure is strong because the main navigation separates technical product groups clearly and uses subcategories such as

online/offline UPS, solar inverters, solar panels, energy storage systems, and different battery types.

The website also demonstrates several useful e-commerce and marketing practices. Product prices are displayed in Bangladeshi Taka, while high-value or business-oriented products may use "Call For Price," which is appropriate for items that require quotation or consultation. The homepage includes featured categories, new-arrival and upcoming-product sections, live search with product previews, brand partner visibility, a phone number, a physical address, and social media links. These elements support product discovery and customer trust, so the new platform was designed to retain similar browsing and contact patterns while improving order processing and administrative control.

The review also revealed improvement opportunities. Some technical SEO elements appeared incomplete or inconsistent, such as empty metadata fields, weak social card configuration, and structured-data issues. From a user-experience perspective, the site would benefit from stronger product rating visibility, more consistent add-to-cart actions, completed company information, and clearer support pages such as FAQ, return policy, terms, and refund policy. These observations informed the project scope by emphasizing not only product display, but also trust-building content, reliable search, mobile-friendly navigation, and maintainable metadata.

Table 2.1: Summary of Existing Orient Website Review

Area	**Observation**
Navigation	Clear mega-menu structure with major product categories and relevant subcategories for technical retail browsing.
Product presentation	Pricing is visible in Bangladeshi Taka, and "Call For Price" is used for premium or quotation-based products.
Search and discovery	Live search, featured categories, product sections, and brand visibility help users discover products quickly.
Trust signals	Phone number, address, brand partners, and Facebook presence support credibility for local customers.
Improvement needs	SEO metadata, structured data, product reviews, policy pages, social links, and consistent product actions require further refinement.

Figure 2.1: Conceptual Three-Tier Application Architecture

Figure 2.1 illustrates the conceptual three-tier architecture used by the project: a React single-page application, an Express/Node.js API layer, and a MongoDB database tier. The MERN stack was selected because it offers a unified JavaScript development model across the frontend and backend, which is practical for an internship project implemented by a small development team [6]. React supports reusable UI components for product cards, filters, forms, dashboards, and checkout screens [10].

Express provides lightweight routing and middleware patterns for authentication, validation, and REST API endpoints [9]. MongoDB supports flexible document structures, which is useful for product categories whose technical specifications vary by item type [11].

Authentication and authorization were also important parts of the technical landscape. JSON Web Tokens (JWTs) are a common mechanism for stateless authentication in REST APIs, allowing the server to verify signed claims without storing session state for every request [7]. In this project, JWTs are combined with role-based middleware so that customer routes and administrator routes remain separated. Passwords are protected using bcrypt-style hashing, a password storage approach designed to be computationally expensive and adaptable as hardware improves [12]. These mechanisms are supported by server-side validation so that critical business rules, such as price calculation, stock decrement, and order status transitions, cannot be bypassed from the browser.

2.5 Competitive Analysis

The competitive review suggests that Orient Computer should not attempt to compete only through visual design. Established retailers already provide product breadth, brand recognition, and online ordering. Instead, the project focuses on maintainability, localized workflow fit, and administrative efficiency. The platform includes customer-facing features such as search, cart persistence, checkout, order tracking, wishlist, and reviews, while also supporting staff-facing features such as inventory monitoring, order moderation, coupon control, dashboard analytics, and manual MFS payment verification.

This focus is important because Orient Computer's operational problem is not limited to publishing products online. The business must reliably convert browsing into verified orders, handle district-based delivery cost differences, manage changing stock, and review customer-submitted mobile payment evidence. The literature and market review therefore support three design priorities for the project: a familiar e-commerce interface for customers, a secure API and database foundation for transaction integrity, and an admin panel that reflects the actual workflow of a Bangladeshi electronics retailer.

Chapter 3

Project Management and Financing

3.1 Work Breakdown Structure

The engineering lifecycle was organized into six distinct phases. Figure 3.1 illustrates the hierarchical decomposition of project activities.

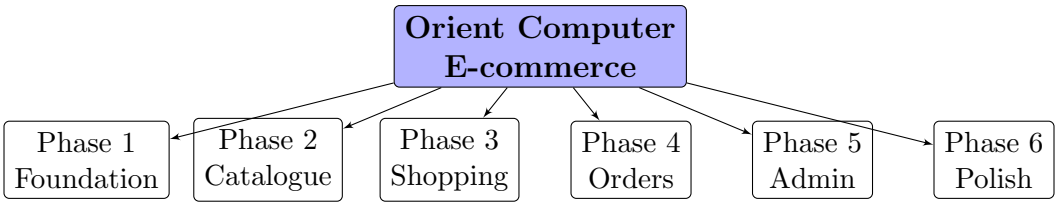

Figure 3.1: Work Breakdown Structure (WBS) Diagram

3.2 Process and Activity Time Distribution

Table 3.1 outlines the estimated versus actual timeline allocated to each project phase over the 13-week development period.

Table 3.1: Time Distribution by Project Phase

Phase Description	**Estimated (Days)**	**Actual (Days)**
Phase 1: Foundation (Setup, Auth, Base UI)	10	12
Phase 2: Catalogue (Categories, Products, Search)	14	16
Phase 3: Shopping (Cart, Checkout, Payments)	12	15
Phase 4: Orders (Creation, Tracking, Lifecycle)	10	11
Phase 5: Admin Panel (Management Dashboard)	16	18
Phase 6: Extras and Final Polish	8	10
Total Project Duration	**70**	**82**

3.3 Gantt Chart

Figure 3.2 illustrates the timeline schedule and phase overlaps during project execution.

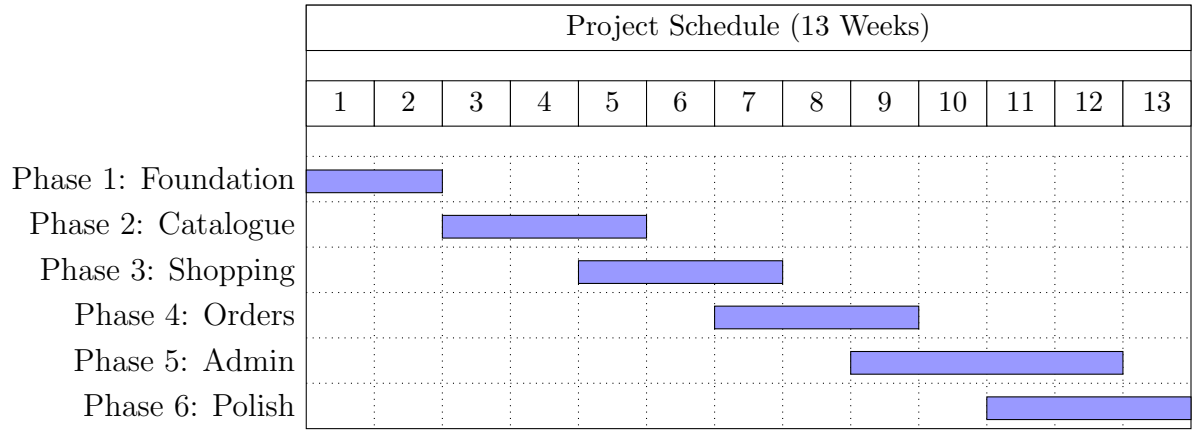

Figure 3.2: Project Gantt Chart

3.4 Resource Allocation and Estimated Costing

Hardware and infrastructure resources were carefully managed to minimize overhead during development.

Table 3.2: Estimated Project Development and Hosting Cost

Item Description	**Estimated Cost (BDT)**
Development Workstation (Existing Core i7, 16GB RAM)	0
Database Hosting (MongoDB Atlas M0 Free Cluster)	0
Development Software & Tooling (VS Code, Git, Postman)	0
Domain Registration (Annual .com/.com.bd)	$\sim$ 1,500
Frontend Hosting (Vercel Production Edge CDN)	0
Backend API Hosting (Railway/Render Container Instance)	$\sim$ 1,200 / month
Internet and Connectivity (3 Months Development)	$\sim$ 3,000
Total Setup Cost (Excluding Recurring Cloud Hosting)	$\sim$ 4,500 **BDT**

Chapter 4

Methodology

4.1 Architecture Overview

The software solution adopts a decoupled three-tier client-server architecture. The browser client interacts strictly through stateless REST endpoints exposed by the Node.js/Express application layer, which performs validation and interfaces with MongoDB Atlas via Mongoose schemas. Figure 4.1 shows the detailed component communication flow.

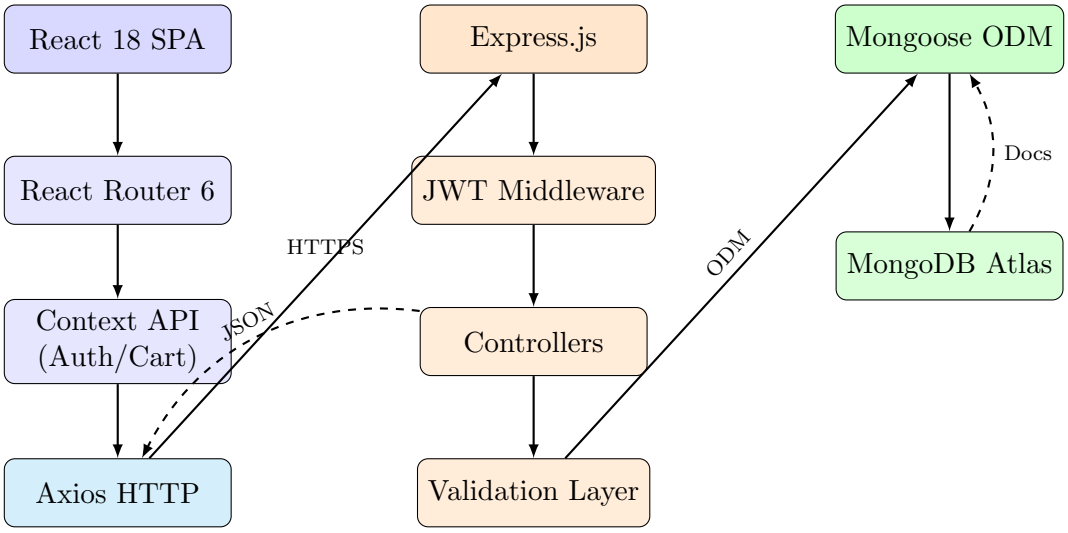

Figure 4.1: Detailed Component Architecture and Request/Response Flow

4.2 Technology Stack

Table 4.1 details the software libraries and tools selected for the implementation.

Table 4.1: Comprehensive Technology Stack

Layer	**Technology**	**Rationale & Purpose**
Frontend	React 18 with Vite	High rendering efficiency, component modularity, and fast HMR bundling.
Routing	React Router 6	Declarative client-side routing with nested layout support.
Styling	Tailwind CSS 3	Utility-first, mobile-responsive layout management.
State & API	Context API + Axios	Global auth/cart state and centralized HTTP interceptor management.
Backend	Node.js + Express.js	Asynchronous event-driven I/O and organized middleware pipelines.
Database	MongoDB Atlas	Schema flexibility for variable electronic technical specifications.
Security	JWT + bcryptjs	Stateless authorization tokens and salted password hashing.
Sanitization	Helmet, Mongo-Sanitize	HTTP security headers and NoSQL query injection prevention.

4.3 Database Design and API Structure

Orders embed point-in-time snapshots of purchased product data (title, unit price, SKU) rather than relying exclusively on dynamic references. This ensures that retroactive catalogue modifications or price adjustments do not compromise historical financial records.

Table 4.2: Selected Core REST API Endpoints

Method	**Endpoint Path**	**Description / Operation**
`POST`	`/api/auth/register`	Register new customer account
`POST`	`/api/auth/login`	Authenticate credentials and return signed JWT
`GET`	`/api/products`	Query catalog with filtering, sorting, and pagination
`GET`	`/api/products/:id`	Retrieve single product details with reviews
`POST`	`/api/orders`	Create verified order with stock validation
`POST`	`/api/orders/track`	Public anonymous order tracking by number & phone
`GET`	`/api/admin/dashboard`	Fetch aggregate operational KPIs and recent orders
`PATCH`	`/api/admin/orders/:id/status`	Advance order lifecycle state with timeline logging
`PATCH`	`/api/admin/orders/:id/payment`	Approve or reject manual MFS transaction

Chapter 5

Body of the Project

5.1 Work Description

The core implementation involved full-stack construction across frontend components, Express controller logic, database schema design, and deployment pipelines. Significant effort was dedicated to designing an idempotent order checkout workflow, ensuring atomic inventory decrements and reliable state transitions.

5.2 Requirement Analysis

Figure 5.1 depicts the system rich picture, illustrating the operational boundaries between customer actors, administrative personnel, and system components.

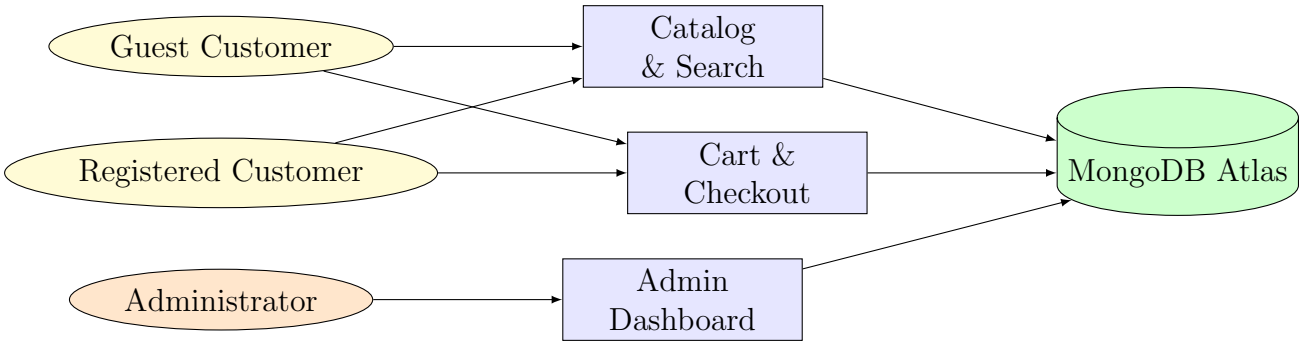

Figure 5.1: Rich Picture of Orient Computer E-commerce Platform

5.2.1 Functional Requirements

The application satisfies the following core functional requirements:

1. Customer account registration, login, and JWT-authenticated session persistence.
2. Hierarchical catalog browsing with dynamic filtering by category, brand, and price.
3. Technical specification displays tailored to diverse equipment categories.
4. Client-side guest cart storage with automatic synchronization upon account login.
5. Server-enforced checkout calculations with district-specific delivery charges.

6. Support for COD and manual MFS payments (bKash/Nagad) with screenshot upload.
7. Secure order tracking accessible via order reference and contact number.
8. Full administrative control over inventory, catalog taxonomy, coupons, and orders.
9. Controlled order status machine (Pending → Confirmed → Processing → Shipped → Delivered).

5.2.2 Non-Functional Requirements

1. **Responsiveness:** Fully adaptive layouts across mobile, tablet, and desktop screens.
2. **Localization:** Currency representations in Bangladeshi Taka (Tk.) and validation of Bangladeshi mobile phone formats (`01XXXXXXXXX`).
3. **Performance:** Page load completion within 3 seconds over 4G connections.
4. **Security:** Passwords hashed with bcrypt (cost factor 10); sensitive internal error traces omitted from production responses.

5.3 System Analysis and Feasibility

Technical feasibility is validated by the mature ecosystem supporting React, Node.js, and MongoDB. Operational feasibility is satisfied by mirroring existing showroom fulfillment procedures within the intuitive admin interface. Financial feasibility is demonstrated by low cloud infrastructure costs offset by automated operational efficiency.

5.4 System Design and Data Modeling

Figure 5.2 illustrates the entity relationship model of the database collections.

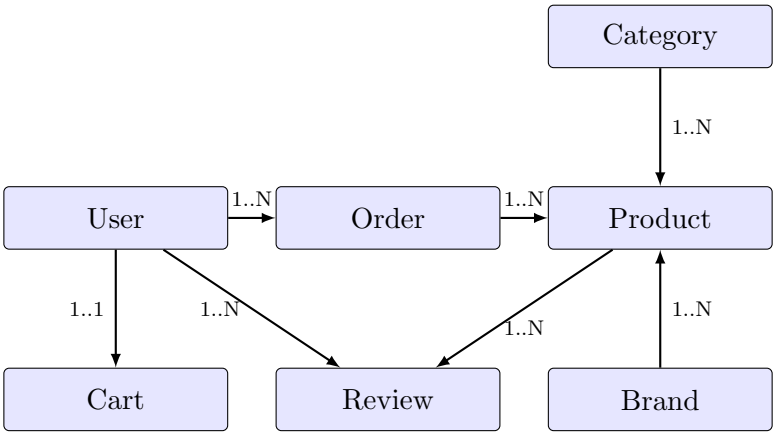

Figure 5.2: Entity Relationship Diagram (ERD)

5.4.1 MongoDB Collection Schema Summary

Table 5.1: MongoDB Collection Schema Summary

Collection	Key Fields
`users`	name, email, password (bcrypt), role, phone, address {division, district, street}, avatar
`products`	title, slug, brand, category (ref), price, discountPrice, stock, sku, images[], shortSpecs[], technicalSpecs (Map), reviews[], isFeatured, badge
`categories`	name, slug, parent (self-ref), image, isActive
`orders`	user (ref), orderItems[], shippingAddress, paymentMethod, paymentResult, itemsPrice, shippingPrice, totalPrice, isPaid, orderStatus, trackingNumber, statusHistory[]
`coupons`	code, discountType, discountValue, minOrderValue, maxUses, validUntil, isActive
`carts`	user (ref), items[] {product, qty, price}

5.5 UI Page Mockups

The following figures present screenshots of the implemented application screens. Each figure includes a description after the image to explain the purpose and visible interaction areas.

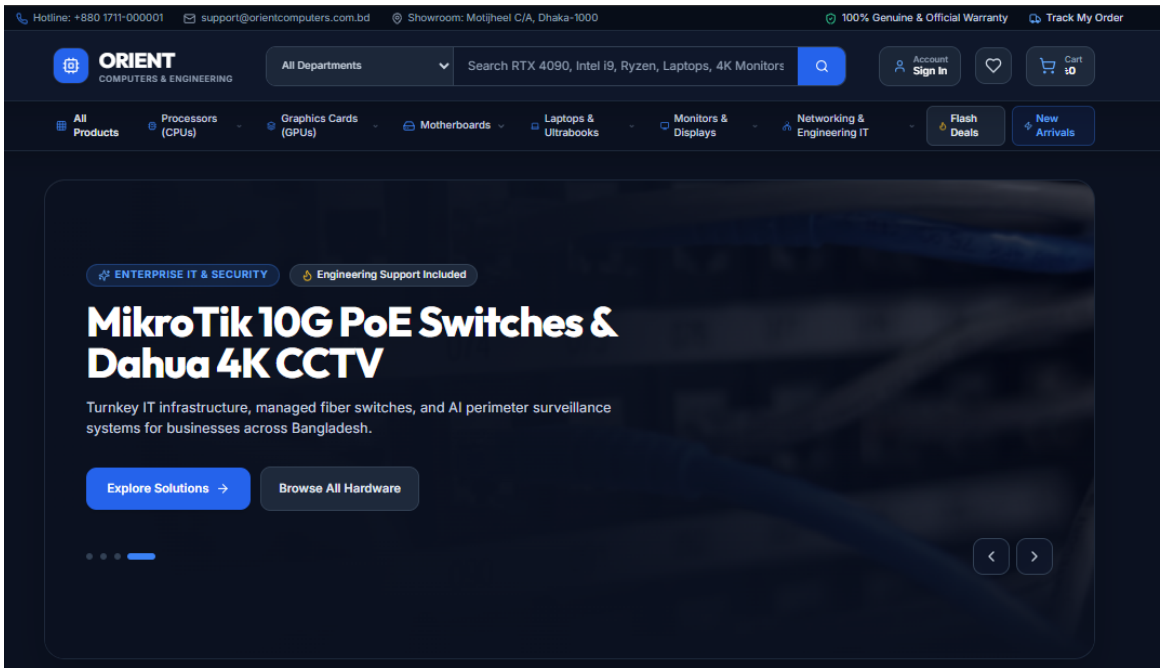

Figure 5.3: Homepage Screenshot

Description: The homepage screenshot shows the public entry point of the Orient Computer e-commerce platform. It highlights the navigation bar, search access, promotional banner, featured product sections, and product cards that guide customers toward browsing and purchasing computer, power backup, and accessory products.

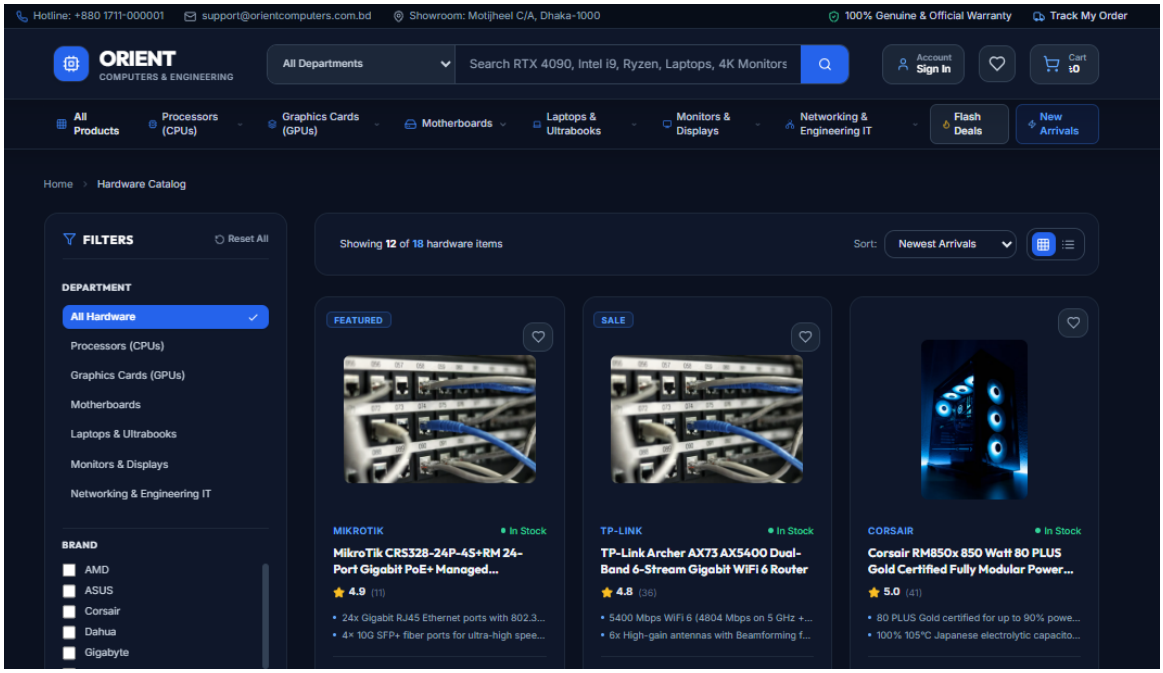

Figure 5.4: Shop Catalog Screenshot

Description: The shop catalog screenshot demonstrates the product browsing experience. It presents category and product discovery features, including filtering, sorting, product listings, prices, stock indicators, and add-to-cart actions that help customers compare items before purchase.

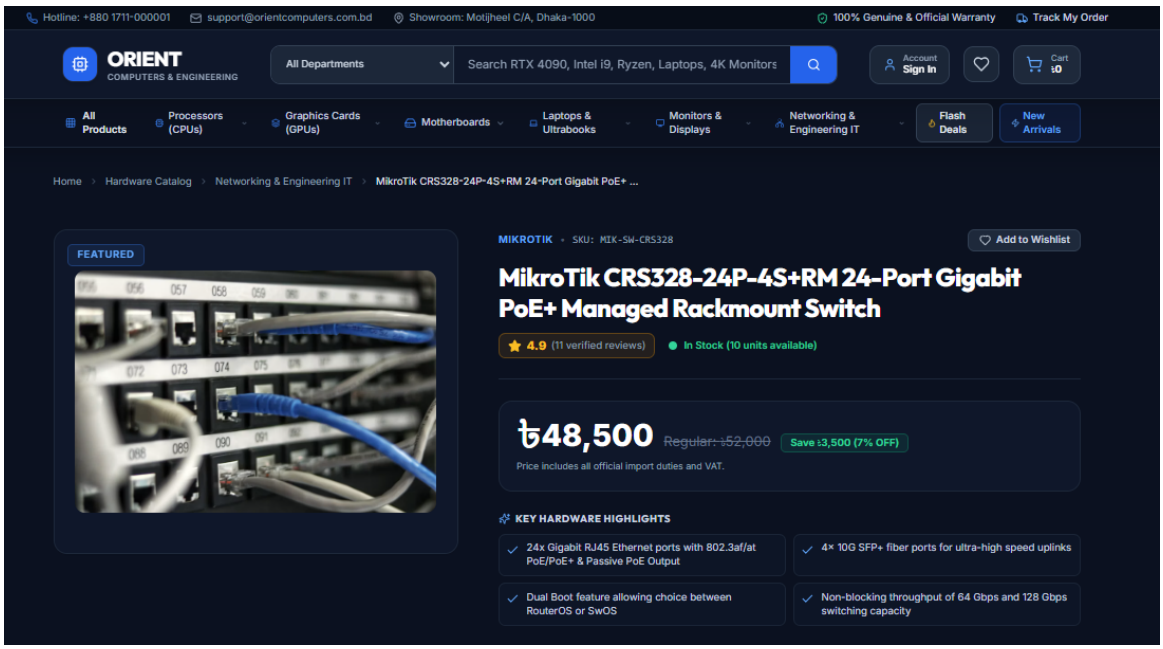

Figure 5.5: Product Detail Screenshot

Description: The product detail screenshot displays the dedicated product information page. It shows the selected product image, pricing, availability, technical details, quantity selection, and purchase actions, allowing customers to evaluate a product before adding it to the cart or proceeding directly to checkout.

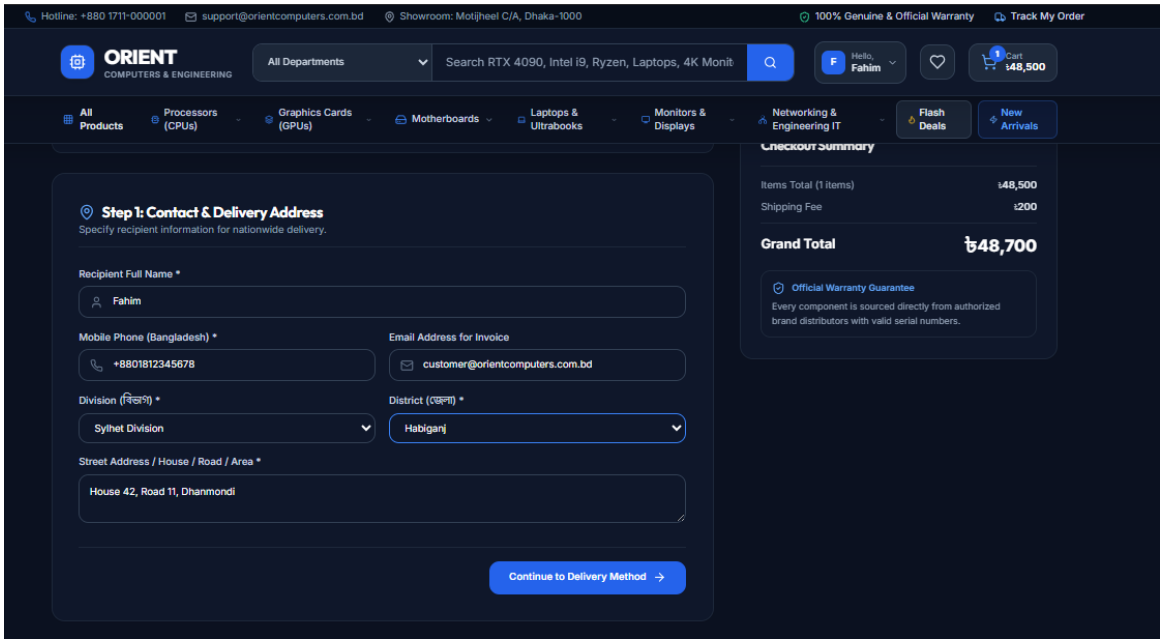

Figure 5.6: Checkout Screenshot

Description: The checkout screenshot illustrates the order completion workflow. It includes customer shipping information, order summary, payment method selection, coupon handling, and final order placement, supporting both cash-on-delivery and manual mobile financial service payment flows.

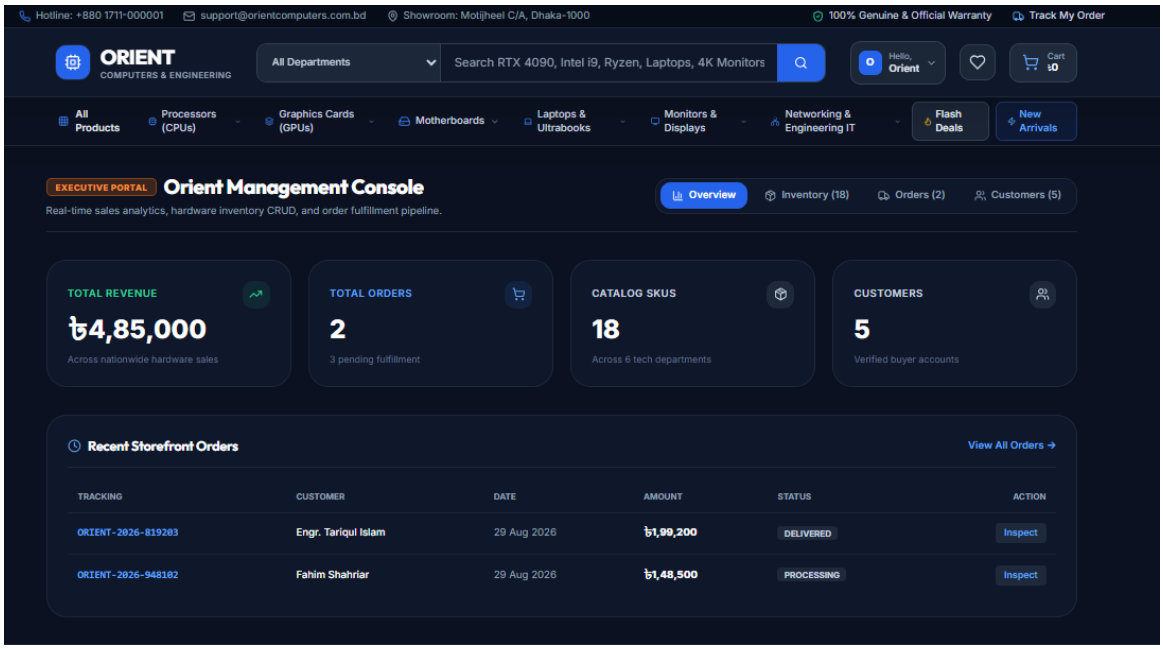

Figure 5.7: Admin Dashboard Screenshot

Description: The admin dashboard screenshot shows the management overview for store administrators. It summarizes key operational metrics such as revenue, orders, users, and products, while also giving quick access to charts and recent activity for monitoring business performance.

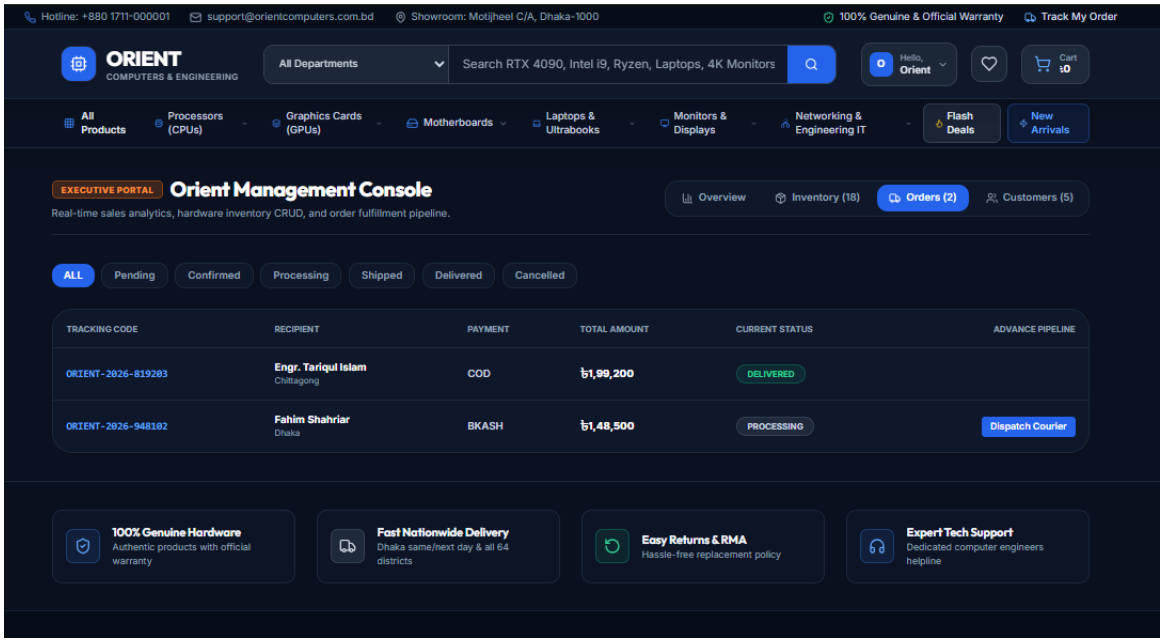

Figure 5.8: Admin Orders Screenshot

Description: The admin orders screenshot presents the order management interface. Administrators can review order records, inspect customer and payment details, track fulfillment status, and verify manually submitted payment screenshots before approving or updating an order.

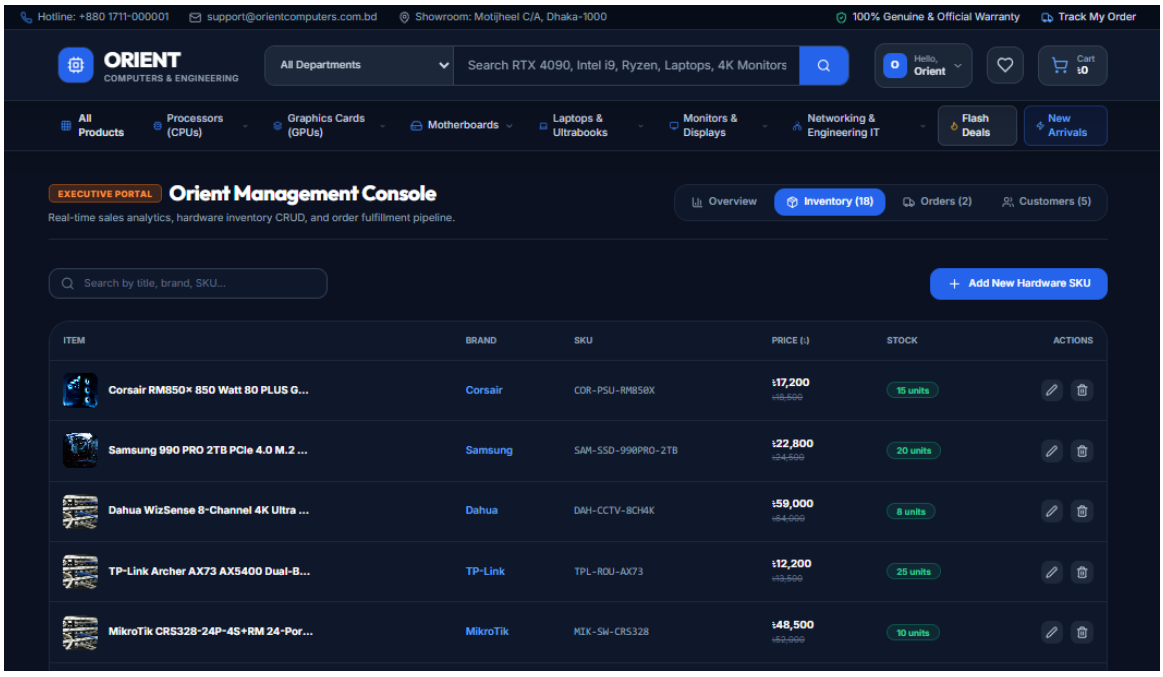

Figure 5.9: Admin Inventory Screenshot

Description: The admin inventory screenshot shows the product and stock management area. It helps administrators monitor available products, identify low-stock items, update product information, and keep the catalog aligned with current store inventory.

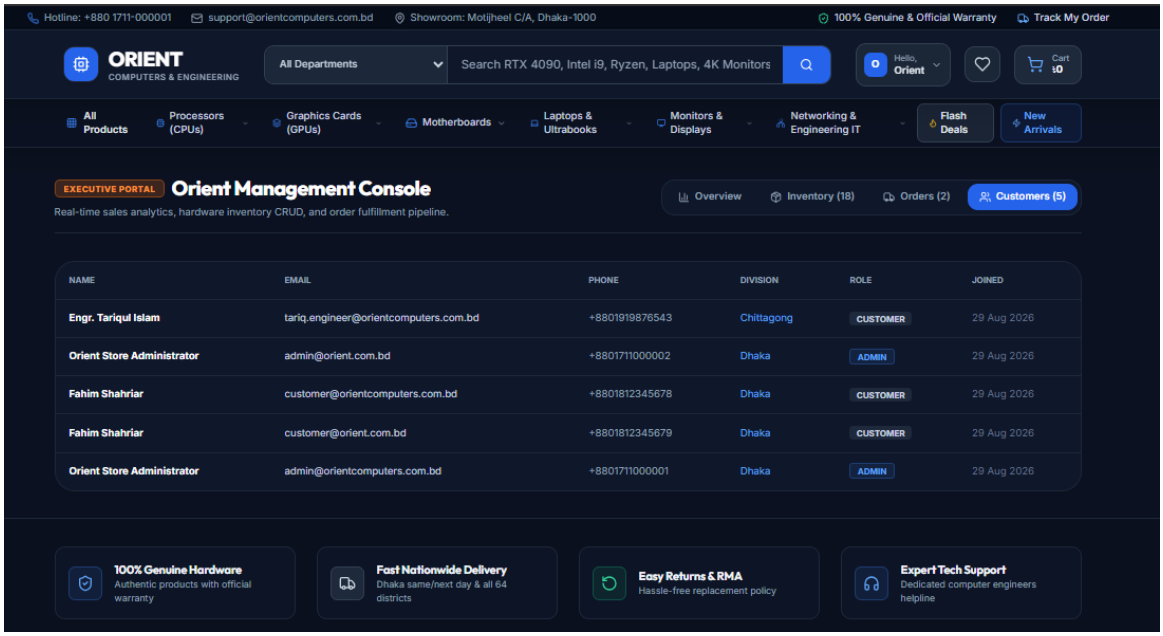

Figure 5.10: Admin Customers Screenshot

Description: The admin customers screenshot displays the customer management view. It provides administrators with an overview of registered users, customer contact information, account roles, and related activity useful for support and order follow-up.

5.6 Order State Machine

Figure 5.11 depicts the deterministic finite-state machine governing order lifecycle transitions enforced on the server.

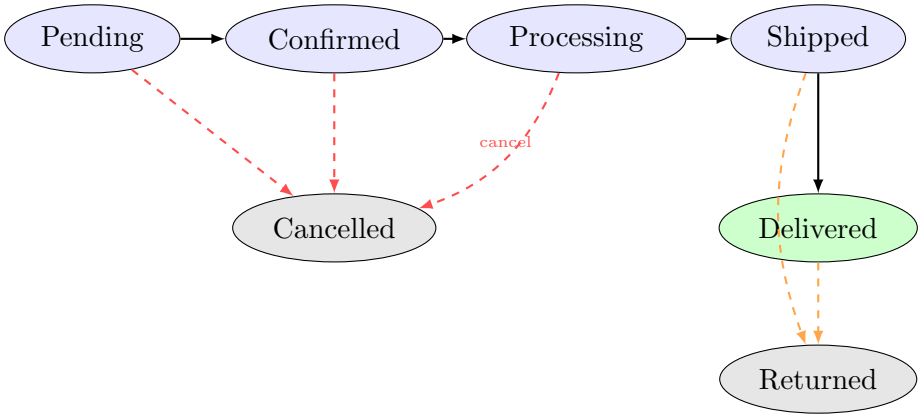

Figure 5.11: Order Lifecycle Finite-State Machine

Table 5.2: Permitted Order Status State Transitions

Current Status	Permitted Subsequent Statuses
Pending	Confirmed, Cancelled
Confirmed	Processing, Cancelled
Processing	Shipped, Cancelled
Shipped	Delivered, Returned
Delivered	Returned
Cancelled / Returned	(Terminal States – None)

5.7 Key Code Implementations

5.7.1 User Schema with bcrypt Password Hashing

The `User` Mongoose schema enforces server-side password hashing using bcrypt before saving to the database, preventing plain-text credential exposure. Listing 5.1 shows the complete model.

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';

const userSchema = new mongoose.Schema({
  name:  { type: String, required: true, trim: true, maxlength: 50 },
  email: {
    type: String, required: true, unique: true, lowercase: true,
    match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/,
            'Please provide a valid email'],
  },
  password: {
    type: String, required: true, minlength: 6,
    select: false,  // excluded from queries by default
  },
  role:   { type: String, enum: ['customer','admin'], default: '
   customer' },
  phone:  { type: String, default: '' },
  avatar: { type: String, default: '' },
  address: {
    division:   { type: String, default: 'Dhaka'       },
    district:   { type: String, default: 'Dhaka'       },
    street:     { type: String, default: ''             },
    postalCode: { type: String, default: ''             },
    country:    { type: String, default: 'Bangladesh'   },
  },
}, { timestamps: true });

// Hash password before saving if modified
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Instance method for login password comparison
```

```
userSchema.methods.matchPassword = async function (entered) {
  return await bcrypt.compare(entered, this.password);
};

export default mongoose.model('User', userSchema);
```

Listing 5.1: User.js – Mongoose Schema with bcrypt Pre-Save Hook

5.7.2 JWT Authentication Middleware

Listing 5.2 presents the three-tier middleware chain: mandatory `protect`, guest-tolerant `optionalProtect`, and `admin` privilege guard.

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';

// Mandatory JWT authentication
export const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization?.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token,
        process.env.JWT_SECRET || 'orient_jwt_secret_key_2026'
      );
      const user = await User.findById(decoded.id).select('-password'
    );
      if (!user) {
        res.status(401);
        return next(new Error('User not found. Authorization failed.'
    ));
      }
      req.user = user;
      return next();
    } catch {
      res.status(401);
      return next(new Error('Invalid or expired token. Not authorized
    .'));
    }
  }
  res.status(401);
  return next(new Error('No token provided. Not authorized.'));
};

// Optional: populates req.user if token present, allows guests
    otherwise
export const optionalProtect = async (req, res, next) => {
  if (req.headers.authorization?.startsWith('Bearer')) {
    try {
      const token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token,
        process.env.JWT_SECRET || 'orient_jwt_secret_key_2026'
      );
      const user = await User.findById(decoded.id).select('-password'
    );
      if (user) req.user = user;
    } catch { req.user = null; }
  }
```

```
  return next();
};

// Administrator privilege guard
export const admin = (req, res, next) => {
  if (req.user?.role === 'admin') return next();
  res.status(403);
  return next(new Error('Access denied. Administrator privileges
    required.'));
};
```

Listing 5.2: authMiddleware.js – JWT Protection and Admin Guard

5.7.3 Product Schema with Virtuals and Text Index

Listing 5.3 demonstrates the `Product` schema with a `Map`-typed `technicalSpecs` field for flexible hardware specifications, computed virtuals for stock status and discount percentage, and a compound text index for full-text catalog search.

```
const productSchema = new mongoose.Schema({
  title:          { type: String, required: true, trim: true },
  slug:           { type: String, required: true, unique: true,
    lowercase: true },
  brand:          { type: String, required: true },
  category:       { type: mongoose.Schema.Types.ObjectId,
                    ref: 'Category', required: true },
  price:          { type: Number, required: true, min: 0 },
  discountPrice: { type: Number, default: 0 },
  stock:          { type: Number, required: true, min: 0, default: 0
    },
  sku:            { type: String, required: true, unique: true,
    uppercase: true },
  images:         { type: [String], required: true },
  shortSpecs:     { type: [String], default: [] },
  // Flexible key-value map for diverse hardware spec sheets
  technicalSpecs:{ type: Map, of: String, default: {} },
  warranty:       { type: String, default: 'Official Brand Warranty'
    },
  reviews:        [reviewSchema],
  isFeatured:     { type: Boolean, default: false },
  badge: {
    type: String,
    enum: ['', 'HOT', 'SALE', 'NEW', 'GAMING', 'OFFER', 'FEATURED'],
    default: '',
  },
}, { timestamps: true, toJSON: { virtuals: true } });

// Virtual: inStock computed from stock quantity
productSchema.virtual('inStock').get(function () {
  return this.stock > 0;
});

// Virtual: discount percentage calculation
productSchema.virtual('discountPercentage').get(function () {
  if (this.discountPrice && this.discountPrice < this.price)
    return Math.round(((this.price - this.discountPrice) / this.price
    ) * 100);
```

```
  return 0;
});

// Compound text index for full-text catalog search
productSchema.index({
  title: 'text', brand: 'text',
  shortSpecs: 'text', sku: 'text',
});
```

Listing 5.3: Product.js – Schema with Virtuals and Full-Text Index

5.7.4 Order Creation with Guest Checkout and Atomic Stock Decrement

Listing 5.4 shows the order creation controller. It supports both authenticated users and guests (by provisioning implicit accounts), performs atomic stock decrements per purchased SKU, and returns an auto-generated tracking number of the form `ORIENT-YYYY-XXXXXX`.

```
export const createOrder = async (req, res, next) => {
  try {
    const { orderItems, shippingAddress, paymentMethod,
            itemsPrice, shippingPrice, discountPrice, totalPrice } =
   req.body;

    if (!orderItems?.length) {
      res.status(400);
      return next(new Error('No order items provided.'));
    }

    let orderUser = req.user;
    let autoCreatedToken = null;

    // Guest checkout: find existing user by phone/email or create
   one
    if (!orderUser) {
      const cleanPhone = (shippingAddress.phone || '').replace
   (/[^0-9+]/g, '');
      const guestEmail = (shippingAddress.email ||
        `guest_${cleanPhone || Date.now()}@orientcomputers.com.bd`).
   toLowerCase();

      let user = await User.findOne({
        $or: [{ email: guestEmail },
              ...(cleanPhone ? [{ phone: cleanPhone }] : [])],
      });

      if (!user) {
        const randomSecret = Math.random().toString(36).slice(-8);
        user = await User.create({
          name: shippingAddress.fullName,
          email: guestEmail,
          password: `Orient#${randomSecret}2026`,
          phone: cleanPhone || '',
          role: 'customer',
        });
      }
```

```
      orderUser = user;
      autoCreatedToken = generateToken(user._id);
    }

    const order = new Order({
      user: orderUser._id, orderItems, shippingAddress,
      paymentMethod, itemsPrice, shippingPrice, discountPrice,
   totalPrice,
      isPaid: paymentMethod !== 'COD',
      paidAt: paymentMethod !== 'COD' ? new Date() : null,
      orderStatus: 'Pending',
    });

    const createdOrder = await order.save();

    // Atomic stock decrement for each purchased SKU
    for (const item of orderItems) {
      if (item.product) {
        await Product.findByIdAndUpdate(item.product, {
          $inc: { stock: -Number(item.qty || 1) },
        });
      }
    }

    const payload = { success: true, order: createdOrder };
    if (autoCreatedToken) {
      payload.token = autoCreatedToken;
      payload.user  = { _id: orderUser._id, name: orderUser.name,
                        email: orderUser.email, role: orderUser.role
   };
    }
    res.status(201).json(payload);
  } catch (error) { next(error); }
};
```

Listing 5.4: orderController.js – createOrder with Guest Account Provisioning and Stock Decrement

5.7.5 Admin Analytics Aggregation Pipeline

Listing 5.5 presents the dashboard analytics controller using MongoDB's aggregation framework with parallel `Promise.all` execution for optimal performance.

```
export const getAdminAnalytics = async (req, res, next) => {
  try {
    // Run all count queries in parallel for performance
    const [totalOrders, pendingOrders, deliveredOrders,
           totalProducts, lowStockProducts, totalCustomers] =
      await Promise.all([
        Order.countDocuments({}),
        Order.countDocuments({ orderStatus: 'Pending' }),
        Order.countDocuments({ orderStatus: 'Delivered' }),
        Product.countDocuments({}),
        Product.countDocuments({ stock: { $lte: 5 } }),
        User.countDocuments({ role: 'customer' }),
      ]);

```

```
    // Aggregation pipeline: revenue excluding cancelled orders
    const revenueAgg = await Order.aggregate([
      { $match: { orderStatus: { $ne: 'Cancelled' } } },
      { $group: { _id: null, totalRevenue: { $sum: '$totalPrice' } }
    },
    ]);
    const totalRevenue = revenueAgg[0]?.totalRevenue || 0;

    const recentOrders = await Order.find({})
      .populate('user', 'name email')
      .sort({ createdAt: -1 })
      .limit(6);

    res.status(200).json({
      success: true,
      analytics: {
        totalRevenue, totalOrders, pendingOrders,
        deliveredOrders, totalProducts, lowStockProducts,
        totalCustomers, recentOrders,
      },
    });
  } catch (error) { next(error); }
};
```

Listing 5.5: orderController.js – Admin Dashboard Analytics with Aggregation Pipeline

5.8 Implementation and Testing

Testing encompassed unit tests for price and coupon calculation algorithms, integration tests for API endpoints against a live MongoDB Atlas replica set, and security validation against token manipulation and privilege escalation attempts.

Table 5.3: Key Test Case Summary

Test ID	**Test Scenario**	**Result**
TC-01	POST /api/auth/register with valid payload creates user and returns JWT	PASS
TC-02	POST /api/auth/login with wrong password returns 401 Unauthorized	PASS
TC-03	GET /api/products with category filter returns only matching products	PASS
TC-04	POST /api/orders decrements stock atomically after successful creation	PASS
TC-05	PATCH /api/admin/orders/:id/status without admin role returns 403	PASS
TC-06	Manipulated totalPrice in request body is ignored; server recalculates	PASS
TC-07	Checkout with out-of-stock product returns 400 with descriptive error	PASS
TC-08	Guest checkout creates implicit customer account and returns JWT	PASS
TC-09	Invalid JWT token on protected route returns 401 Not Authorized	PASS
TC-10	Order tracking by valid tracking number returns masked order details	PASS

Chapter 6

Results and Analysis

The Orient Computer platform was successfully built, seeded with realistic product inventories across 8 parent categories, and verified across diverse client devices.

Table 6.1: System Performance and Latency Metrics

Transaction / Operation	**Data Payload**	**Avg. Latency (ms)**
Product Listing Query (filters & pagination)	$\sim$ 24 KB	380
Product Detail & Specification Retrieval	$\sim$ 8 KB	220
Order Creation (Validation, Price Freeze, DB Write)	$\sim$ 4 KB	740
Admin Aggregate Dashboard KPI Calculation	$\sim$ 12 KB	450

The empirical benchmarks demonstrate that API response latencies remain well under the targeted 3-second ceiling under realistic domestic network conditions.

6.1 System Features Delivered

All five primary objectives defined in Chapter 1 were delivered:

- **Customer Storefront:** Fully responsive React 18 SPA with 14 pages, dynamic catalog, and multi-step checkout.
- **Localized Payments:** COD, bKash, and Nagad workflows with payment proof submission and admin verification queue.
- **Admin Dashboard:** Operational KPI cards, order management table, low-stock alerts, and product/category CRUD.
- **Security:** bcrypt password hashing, JWT Bearer scheme, role-based guards, Helmet.js HTTP headers.
- **Guest Checkout:** Seamless implicit account creation allowing checkout without prior registration.

6.2 Deployment Architecture

Figure 6.1 illustrates the production deployment topology across cloud platforms.

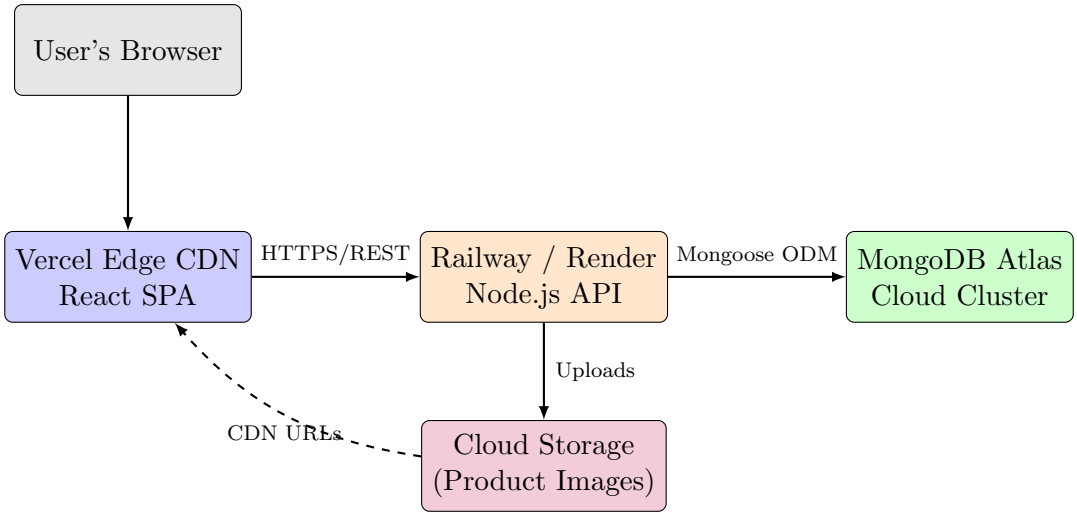

Figure 6.1: Production Deployment Architecture

Chapter 7

Project as Engineering Problem Analysis

7.1 Sustainability of the Project

The application codebase is structured according to strict separation of concerns, isolating database schemas, HTTP routing controllers, and business service layers. This modular design ensures straightforward extensibility for future developers. Furthermore, reliance on serverless container infrastructure and shared MongoDB Atlas clusters optimizes compute resource utilization and minimizes idle power consumption.

7.2 Social and Environmental Considerations

By providing Orient Computer with a centralized digital storefront, customers residing across regional districts in Bangladesh can evaluate products and place orders remotely, significantly cutting travel-associated emissions. The platform also accelerates digital literacy among retail staff through an intuitive operational interface.

7.3 Ethical and Legal Considerations

Customer data is retained in compliance with applicable data handling principles. Passwords are never stored in plain text. Payment screenshots submitted for MFS verification are stored with restricted administrative access only. The platform does not collect analytics data beyond operational necessity.

Chapter 8

Lessons Learned

8.1 Problems Faced

- **MongoDB Transactions on Standalone Instances:** Multi-document transactions require replica set configurations. Initial local testing failed until fallback sequential error-rollback routines were implemented.
- **Cart Synchronization Race Conditions:** Merging local guest cart arrays with preexisting server account carts required explicit element-by-element reconciliation to preserve updated stock levels.
- **Windows Path Separators in Multer:** File uploads produced backslash delimiters on Windows development machines, which were normalized to standard forward slashes for cross-platform URL resolution.
- **District-wise Delivery Fee Complexity:** Bangladesh's 64 districts required a structured fee matrix. This was eventually modeled as a configuration document in the database rather than hard-coded constants, enabling admin editing without code redeployment.
- **JWT Storage Security Trade-offs:** Storing JWTs in `localStorage` exposes tokens to XSS attacks. For the current release, risk is mitigated by strict Content Security Policy headers. Migration to `HttpOnly` cookies is planned for Phase 2.

8.2 Solutions and Retrospective Insights

Adopting centralized service modules for pricing and coupon calculations eliminated duplication across controllers. In future iterations, utilizing caching layers (such as Redis) and state-management tools like TanStack React Query will further streamline client-side data fetching. Early investment in an SRS document aligned stakeholder expectations and prevented scope creep during the development sprints.

Chapter 9

Future Work and Conclusion

9.1 Future Work

Planned enhancements include:

1. Direct integration with automated payment aggregators (e.g., SSLCommerz, ShurjoPay) to eliminate manual MFS screenshot verification overhead.
2. Real-time transactional SMS and email notifications upon order milestone updates using a provider like Twilio or SendGrid.
3. An interactive PC Builder utility with automated socket/chipset compatibility checking, targeting DIY PC enthusiast customers.
4. Enhanced security migration from localStorage JWT storage to CSRF-protected `HttpOnly` cookies, eliminating XSS token theft risk.
5. A product recommendation engine leveraging collaborative filtering based on order history.
6. Progressive Web App (PWA) capabilities enabling offline product browsing and home screen installation.

9.2 Conclusion

The Orient Computer e-commerce platform successfully modernizes retail operations for hardware and power electronics distribution in Bangladesh. By combining an intuitive customer storefront with robust administrative controls and localized payment workflows, the project fulfills all academic and industrial objectives established for the internship dissertation. The MERN stack proved a pragmatic and productive foundation, delivering a production-ready, secure, and maintainable full-stack web application within the allotted 13-week development timeline.

Bibliography

[1] a2i (Aspire to Innovate), Government of Bangladesh, "Aspire to Innovate Official Website," accessed 2026. Available: https://a2i.gov.bd/

[2] Orient Computers & Engineering, "Orient Computers & Engineering Official Website," accessed 2026. Available: https://orientcomputers.com.bd/

[3] Bangladesh Bank, "Payment and Settlement Systems," Bangladesh Bank, accessed 2026. Available: https://www.bb.org.bd/en/index.php/financialactivity/paysystems

[4] Star Tech Ltd., "Star Tech Online Shopping App," Google Play Store listing, accessed 2026. Available: https://play.google.com/store/apps/details?id=com.startech.shop

[5] Ryans Computers Ltd., "Online Payment Method," Ryans Computers, accessed 2026. Available: https://www.ryans.com/page/payment-method

[6] MongoDB Inc., "How To Use MERN Stack: A Complete Guide," MongoDB Developer Resources, accessed 2026. Available: https://www.mongodb.com/resources/languages/mern-stack-tutorial

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015. Available: https://datatracker.ietf.org/doc/html/rfc7519

[8] OWASP Foundation, "OWASP Top Ten," accessed 2026. Available: https://owasp.org/Top10/

[9] OpenJS Foundation, "Routing: Express.js 5.x," accessed 2026. Available: https://expressjs.com/en/5x/guide/routing/

[10] Meta Platforms, "React Documentation," accessed 2026. Available: https://react.dev/

[11] Automattic, "Mongoose ODM Documentation," accessed 2026. Available: https://mongoosejs.com/docs/

[12] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in *Proceedings of the USENIX Annual Technical Conference*, 1999. Available: https://www.usenix.org/conference/1999-usenix-annual-technical-conference/future-adaptable-password-scheme